

DarkNetTracker Project Report

Project Maintenance Update

May 17, 2026

Contents

1	Executive Summary	2
2	Repository Status	3
2.1	Completed in This Update	3
2.2	Current Status Snapshot	3
3	Architecture Update	5
3.1	Runtime Flow	5
3.2	Security Posture	5
4	Implementation Details	6
4.1	Backend	6
4.2	Frontend	6
4.3	Python Engine	7
5	Verification	8
5.1	Checks Run	8
5.2	Known Remaining Gaps	8
6	Conclusion	9

Chapter 1

Executive Summary

DarkNetTracker is a research-oriented traffic-correlation platform with a Python analysis engine, an Express backend, SQLite persistence, and a Next.js dashboard. This update focused on closing the highest-priority production gaps that were identified in the project backlog.

The most important outcome of this pass is that the repository now has a realistic production-hardening baseline instead of documentation that overclaimed completion. Security controls, environment-aware startup behavior, migration support, CI validation, and frontend resilience have been improved and documented.

Chapter 2

Repository Status

2.1 Completed in This Update

- Added a central backend configuration module with development and production settings.
- Added a migration runner and schema migration tracking table for SQLite startup.
- Added production startup enforcement that blocks demo passwords in production mode.
- Replaced ad hoc security headers with `helmet`.
- Tightened CORS to configured origins only.
- Added CSRF token generation and verification for authenticated write routes.
- Hardened upload validation for size and extension checks.
- Added structured request IDs, centralized backend error handling, and graceful shutdown.
- Added Python subprocess retry, timeout, and failure logging improvements.
- Added Pydantic-based runtime validation for Python pipeline inputs.
- Added frontend error and loading boundaries plus accessibility improvements for critical controls.
- Added GitHub Actions CI and refreshed project documentation.
- Added backend API smoke tests and frontend unit test scaffolding with passing test runs.
- Added an admin metrics endpoint and an OpenAPI source endpoint for operations and integration work.

2.2 Current Status Snapshot

Area	Status	Notes
Backend API	Improved	Security middleware, validation, migration runner, graceful shut-down
Frontend Dashboard	Improved	Error boundary, loading state, auth flow updated for CSRF
Python Engine	Improved	Pydantic validation and structured progress logging
Documentation	Improved	README, runbook, troubleshooting, architecture, security checklist
CI/CD	Added baseline	GitHub Actions workflow for installs, migration, tests, frontend build
Testing	Improved	Python validation tests, backend API smoke tests, frontend unit tests, frontend build

Chapter 3

Architecture Update

3.1 Runtime Flow

1. The frontend calls the backend bootstrap endpoint for environment metadata.
2. A user authenticates and receives a bearer token plus a CSRF token.
3. Authenticated analysis requests are validated and persisted as sessions and jobs.
4. The queue launches the Python engine through `engine.api.py`.
5. Results are written to SQLite and broadcast to the frontend over WebSocket.
6. Export endpoints produce CSV, HTML, and PDF outputs from stored session results.

3.2 Security Posture

Control	State	Detail
Default credentials	Improved	Production startup now fails if demo passwords remain configured
CORS restrictions	Implemented	Requests are limited to configured origins
CSRF protection	Implemented	Authenticated non-GET routes require <code>X-CSRF-Token</code>
Security headers	Implemented	<code>helmet</code> is enabled with CSP and related headers
Rate limiting	Implemented	General API, auth, and upload limiters are configured
Upload validation	Implemented	File size and extension checks added before processing
TLS support	Available	Optional HTTPS startup via configured certificate and key paths

Chapter 4

Implementation Details

4.1 Backend

The backend now uses a single configuration source in `backend/config.js`. Startup creates required directories, reads environment settings, and validates production requirements before listening for traffic.

Database initialization was moved behind a migration runner in `backend/migrations.js`. This gives the project a maintainable path for future schema changes instead of one-off `ALTER TABLE` calls embedded directly in runtime setup.

The Express server now includes:

- `helmet`-based security headers
- request IDs
- stricter request validation with Zod
- CSRF enforcement on authenticated write operations
- upload rate limiting and file validation
- centralized error responses
- graceful shutdown for the HTTP server and WebSocket clients
- an admin metrics endpoint at `/api/metrics`
- an OpenAPI source endpoint at `/api/docs/openapi.yaml`

4.2 Frontend

The Next.js dashboard was updated to support the new CSRF flow by storing the CSRF token returned at login and including it automatically on non-GET API requests. The frontend also now has:

- an application error boundary
- an application loading screen
- accessibility labels for important login and admin controls
- baseline unit tests using Vitest and Testing Library

4.3 Python Engine

Runtime validation in `validation.py` now uses a Pydantic model for request shape and boundary enforcement. The pipeline service logs major processing milestones so long runs are easier to trace when launched by the backend queue.

Chapter 5

Verification

5.1 Checks Run

- Frontend production build completed successfully.
- Backend migration bootstrap completed successfully.
- Backend startup smoke test completed successfully.
- Login, authenticated read, and CSRF-protected logout smoke tests completed successfully.
- Python validation test suite completed successfully: 11 tests passed.
- Backend API smoke tests completed successfully: 2 tests passed.
- Frontend unit tests completed successfully: 3 tests passed.
- Python simulate-mode pipeline smoke test completed successfully.

5.2 Known Remaining Gaps

- Broader automated backend API tests are still needed.
- Browser-level end-to-end tests are still needed.
- External monitoring, alerting, and dependency vulnerability cleanup are still pending.
- The broader feature roadmap remains open and should not be described as complete.

Chapter 6

Conclusion

This update moves DarkNetTracker from a mostly working prototype with optimistic documentation toward a more credible production-ready baseline. The codebase now has stronger security defaults, a repeatable database migration path, more reliable subprocess execution, cleaner operational documentation, and a frontend that understands the hardened auth flow.

The project is in a significantly better state for continued development and controlled deployment, but it should still be described as an actively maturing platform rather than a fully finished enterprise system.